q

COURSE CODE/TITLE:	CPE 010/Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	July 23 2026	**DATE SUBMITTED:**	July 23 2026 part 1, July 28 2026 part 2
NAME:	Mendoza, Roy Gabriel	**INSTRUCTOR:**	Engr. JIMLORD QUEJADO

ACTIVITY NO. 3 **LINKED LISTS**

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 3-1. Output of Initial/Simple Implementation

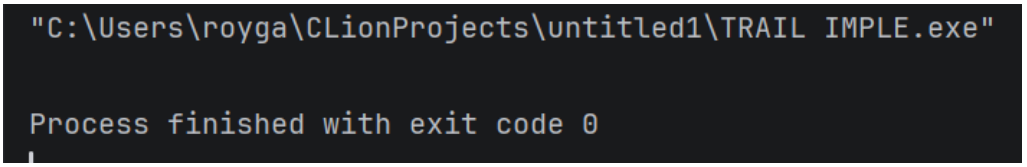
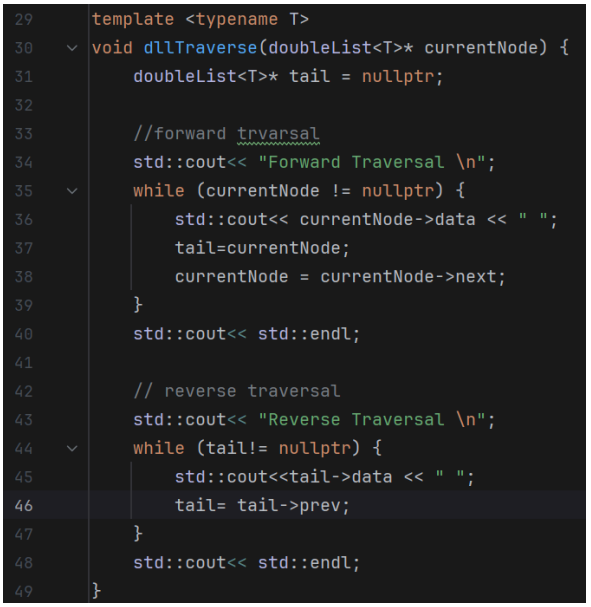
Screenshot	
Discussion	The program was able to create a linked list but not have any of the cout or to print out the output. Since this is a node by node, we could create a looping code that will print out the contents of the nodes to show the output per.

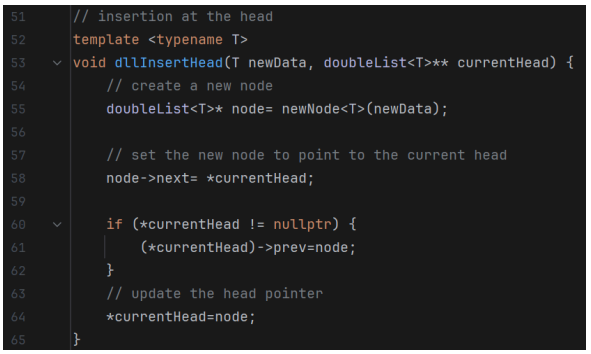
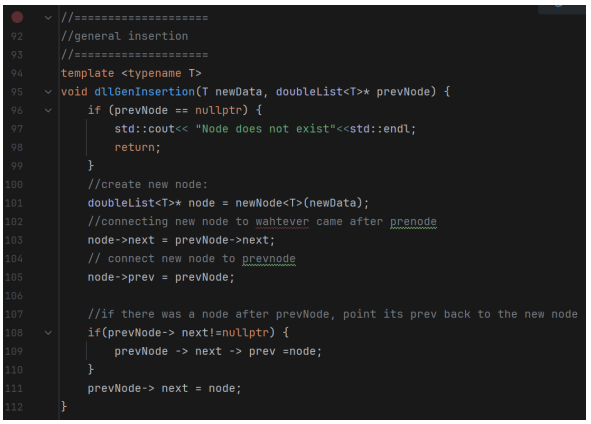
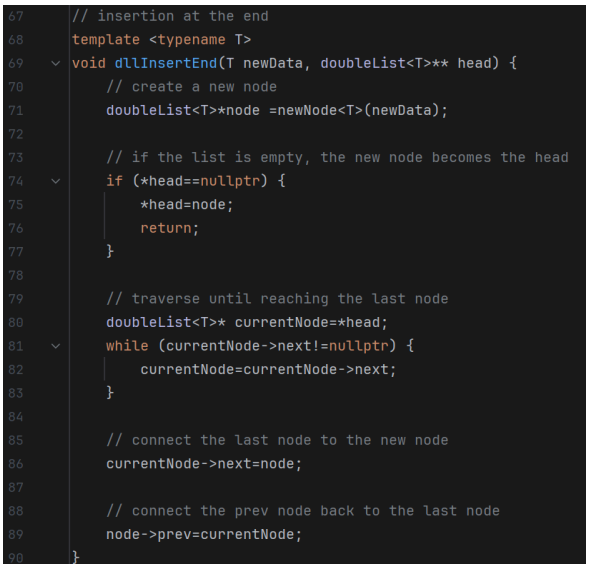
Table 3-2. Code for the List Operations

OPERATION:	SCREENSHOT:
Traversal	

```
template <typename T>
void dllTraverse(doubleList<T>* currentNode) {
    doubleList<T>* tail = nullptr;

    //forward trvarsal
    std::cout<< "Forward Traversal \n";
    while (currentNode != nullptr) {
        std::cout<< currentNode->data << " ";
        tail=currentNode;
        currentNode = currentNode->next;
    }
    std::cout<< std::endl;

    // reverse traversal
    std::cout<< "Reverse Traversal \n";
    while (tail!= nullptr) {
        std::cout<<tail->data << " ";
        tail= tail->prev;
    }
    std::cout<< std::endl;
}
```

Insertion at head	
Insertion at any part of the list	
Insertion at the end	

```cpp
// insertion at the head
template <typename T>
void dllInsertHead(T newData, doubleList<T>** currentHead) {
    // create a new node
    doubleList<T>* node= newNode<T>(newData);

    // set the new node to point to the current head
    node->next= *currentHead;

    if (*currentHead != nullptr) {
        (*currentHead)->prev=node;
    }
    // update the head pointer
    *currentHead=node;
}
```

```cpp
//====================
//general insertion
//====================
template <typename T>
void dllGenInsertion(T newData, doubleList<T>* prevNode) {
    if (prevNode == nullptr) {
        std::cout<< "Node does not exist"<<std::endl;
        return;
    }
    //create new node:
    doubleList<T>* node = newNode<T>(newData);
    //connecting new node to wahtever came after prenode
    node->next = prevNode->next;
    // connect new node to prevnode
    node->prev = prevNode;

    //if there was a node after prevNode, point its prev back to the new node
    if(prevNode-> next!=nullptr) {
        prevNode -> next -> prev =node;
    }
    prevNode-> next = node;
}
```

```cpp
// insertion at the end
template <typename T>
void dllInsertEnd(T newData, doubleList<T>** head) {
    // create a new node
    doubleList<T>*node =newNode<T>(newData);

    // if the list is empty, the new node becomes the head
    if (*head==nullptr) {
        *head=node;
        return;
    }

    // traverse until reaching the last node
    doubleList<T>* currentNode=*head;
    while (currentNode->next!=nullptr) {
        currentNode=currentNode->next;
    }

    // connect the last node to the new node
    currentNode->next=node;

    // connect the prev node back to the last node
    node->prev=currentNode;
}
```

Deletion of a node	

```
template <typename T>
void dllDelete(T findData, doubleList<T>** head) {
    //stopping if the list is empty
    if(*head == nullptr)return;

    //start from the head
    doubleList<T>* currentNode = *head;

    //search until the value is found or end of list is reached
    while(currentNode != nullptr&&currentNode->data != findData) {
        currentNode = currentNode->next;
    }
    //when the value is not found
    if(currentNode == nullptr)return;

    // if there is a node before the one we're deleting, link it to the node after it
    if(currentNode->prev!=nullptr) {
        currentNode->prev->next = currentNode->next;
        } else{
            //updating the head pointer
            *head = currentNode->next;
    }
    //if there is a node after the one being deleted, link it tot he before to complet
    if(currentNode->next!=nullptr) {
```

```
    }
    //if there is a node after the one being deleted, link it tot he before to complet
    if(currentNode->next!=nullptr) {
        currentNode-> next -> prev = currentNode->prev;
    }
//deletion proepr
    delete currentNode;
}

#endif //UNTITLED1_MENDOZA_ROY_DOUBLY_LL_JULY23_H
```

Table 3-3. Code and Analysis for Singly Linked Lists

In your driver function, show the use of each list operation and show the output in table 3-3 found in section 6 with a descriptive caption for each. The tasks you have to perform are as follows:

a. Traverse the list by passing the head of the created list into the function
b. Insert the element 'G' at the start of the list to replace the current node. Output should now show "GCPE101"
c. Insert an element "E" with the previous node element being "P". Output should now show "GCPEE101".
d. Delete the node containing the element C.
e. Delete the node containing the element P.
f. Show the elements in the list. Output should be "GEE101".

a.

Source Code	`std::cout << "Initial Traversal" << std::endl;` `dllTraverse(head);`

Console	`C:\Users\royga\CLionProjects\untitled1\Mendoza_Roy_Doubly_LL_July23.exe`
	`Initial Traversal`
	`Forward Traversal`
	`C P E 1 0 1`

b.

Source Code

```
std::cout << "Inserting at the Head" << std::endl;
dllInsertHead( newData: 'G', &head);
dllTraverse(head);
```

Console

```
Inserting at the Head
Forward Traversal
G C P E 1 0 1
```

c.

Source Code

```
38      std::cout << "General Insertion (after the head -> next -> next)"<< std::endl;
39      dllGenInsertion( newData: 'E', head->next->next);
40      dllTraverse(head);
```

Console

```
General Insertion (after the head -> next -> next)
Forward Traversal
G C P E E 1 0 1
```

d.

Source Code

```
42      std::cout<<"Deleting the C "<<std::endl;
43      dllDelete( findData: 'C', &head);
44      dllTraverse(head);
```

Console

```
Deleting the C
Forward Traversal
G P E E 1 0 1
```

e.

Source Code

```
46      std::cout<<"Deleting the P "<<std::endl;
47      dllDelete( findData: 'P', &head);
48      dllTraverse(head);
```

	Console	Deleting the P Forward Traversal G E E 1 0 1
f.	**Source Code**	50 std::cout << "Final Traversal" << std::endl; 51 dllTraverse(head);
	Console	Final Traversal Forward Traversal G E E 1 0 1

Table 3-4. Modified Operations for Doubly Linked Lists

Screenshots(s) (1st pic would be the singly, 2nd would be the doubly)	Analysis

template <typename T>
class Singlelist {
public:
T data; // Stores the value of the node
Singlelist<T>* next = nullptr; // Points to the next node
};
 	
template <typename T>
class doubleList {
public:
T data;
doubleList<T>* next = nullptr;
doubleList<T>* prev = nullptr;
};
	On this part, the difference or the modification I have done for it to become a doubly was to set not only a next, but also a previous or **prev** to allow bidirectional traversal.

Final List: G --> E --> E --> 1 --> 0 --> 1	Added a reverse traversal of the output, final output. Since in singly, the output is just from the Start until the end, In the doubly, there is another output that shows the end upto the

```
Forward Traversal
G E E 1 0 1
Reverse Traversal
1 0 1 E E G
```

start.

```
// Create an empty linked list
Singlelist<char> *head = new Singlelist<char>;
Singlelist<char> *second = new Singlelist<char>;
Singlelist<char> *third = new Singlelist<char>;
Singlelist<char> *fourth = new Singlelist<char>;
Singlelist<char> *fifth = new Singlelist<char>;
Singlelist<char> *sixth = new Singlelist<char>;
```

In the singly linked list, each node was created with the use of the **new** operator. After the memory allocation, the pointer values had been assigned separately in the **main** function.

In the doubly linked list, a **newNode** function was created that allocate memory for a new node and store the data or in this case the **char** or the letter in each node.

```
head->data = 'C';
head->next = second;

second->data = 'P';
second->next = third;

third->data = 'E';
third->next = fourth;

fourth->data = '1';
fourth->next = fifth;

fifth->data = '0';
fifth->next = sixth;

sixth->data = '1';
sixth->next = nullptr;
```

```
//cpe101 node setting
doubleList<char>* head = newNode( newData: 'C');
doubleList<char>* second = newNode( newData: 'P');
doubleList<char>* third = newNode( newData: 'E');
doubleList<char>* fourth = newNode( newData: '1');
doubleList<char>* fifth = newNode( newData: '0');
doubleList<char>* sixth = newNode( newData: '1');
```

```
//link the nodes:
head->next = second;
second->prev = head;

second->next = third;
third->prev = second;

third->next = fourth;
fourth->prev = third;

fourth->next = fifth;
fifth->prev = fourth;

fifth->next = sixth;
sixth->prev = fifth;
```

```
void sllDelete(T findData, Singlelist<T>** head)
{
    // Stop if the list is empty
    if (*head == nullptr)
        return;

    // Start searching from the head
    Singlelist<T>* currNode = *head;

    // Previous node starts as nullptr
    Singlelist<T>* prevNode = nullptr;

    // Search until the value is found or end of list is reached
    while (currNode != nullptr && currNode->data != findData)
    {
        prevNode = currNode;
        currNode = currNode->next;
    }
    // Value not found
    if (currNode == nullptr)
        return;
    // If deleting the first node
    if (prevNode == nullptr)
    {
        // Move the head to the second node
        *head = currNode->next;
    }
    else
```

```
    else
    {
        // Skip the current node
        prevNode->next = currNode->next;
    }
    // Free the memory of the deleted node
    delete currNode;
}
```

```
void dllDelete(T findData, doubleList<T>** head) {
    //stopping if the list is empty
    if(*head == nullptr)return;

    //start from the head
    doubleList<T>* currentNode = *head;

    //search until the value is found or end of list is reached
    while(currentNode != nullptr&&currentNode->data != findData) {
        currentNode = currentNode->next;
    }
    //when the value is not found
    if(currentNode == nullptr)return;
```

```
132         // if there is a node before the one we're deleting, link it to the node after it
133         if(currentNode->prev!=nullptr) {
134             currentNode->prev->next = currentNode->next;
135             } else{
136                 //updating the head pointer
137                 *head = currentNode->next;
138         }
139         //if there is a node after the one being deleted, link it tot he before to complete doubly
140         if(currentNode->next!=nullptr) {
141             currentNode-> next -> prev = currentNode->prev;
142         }
143     //deletion proepr
144         delete currentNode;
145     }
```

The difference in the delete function of the singly and the doubly is that, in the singly LL, it only reconnects the **next** after deleting the node, while the doubly reconnects both the **next** and the **prev** to still keep the reverse traversal.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

TASK:

ILO B: Solve given problems utilizing linked lists in C++

Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing

Problem Description:
In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.

We'll take the use case of a music player. It should have following functions supported:

- Create a playlist using multiple songs.
- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).

Here are the steps to solve the problem:

- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.

The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

Supplementary CODE:

Header file:

```
#ifndef UNTITLED1_PLAYLIST_H
#define UNTITLED1_PLAYLIST_H

#include <iostream>
#include <ostream>
#include <string>

//==========================
//Doubly linked list node allocation, with the title and the composer
//==========================
class SongNode{
    public:
    std::string songTitle;
    std::string songComposer;
    SongNode* next = nullptr;
    SongNode* prev = nullptr;
};
//==========================
// creation of the connection of the lists to the prev and to the next
//==========================
```

```
SongNode* createSongNode(std::string title, std::string composer) {
    SongNode* node = new SongNode();
    node->songTitle = title;
    node->songComposer = composer;
    node->next = node;
    node->prev = node;
    return node;
}
//====================
//inserting a new song at the front of the playlist,
//====================
void insSongAtHead(std::string title, std::string composer, SongNode** head) {
    SongNode* node = createSongNode( title,  composer);

    if (*head == nullptr) {
        *head = node;
        return;
    }
    SongNode* tail = (*head)->prev;
    node->next = *head;
```

```
    node->prev = tail;
    tail->next = node;
    (*head)->prev = node;
    *head = node;
}

//=====================
//end insert
//====================
void insSongAtEnd(std::string title, std::string composer, SongNode** head) {
    SongNode* node = createSongNode( title,  composer);
    if (*head == nullptr) {
        *head = node;
        return;
    }
    SongNode* tail = (*head)->prev;
    tail->next = node;
    node->prev = tail;
    node->next = *head;
    (*head)->prev = node;
}

//====================
//song search
//====================
SongNode* findSong(std::string title, SongNode* head) {
    if (head == nullptr) return nullptr;

    SongNode* current = head;
    do {
        if (current->songTitle == title) {
            return current;
        }
        current = current->next;
    } while (current != head);
    return nullptr;
}
//==============
//Insert song middle(like the general insert yeah)
//===============
```

```
void insSongBet(std::string titleToIns, std::string newTitle, std::string newComposer, SongNode** head) {
    SongNode* refNode = findSong( titleToIns, *head);

    if (refNode == nullptr) {
        std::cout << titleToIns << " was not found in the playlist" << std::endl;
        return;
    }
    SongNode* node = createSongNode( newTitle,  newComposer);
    node->next = refNode->next;
    node->prev = refNode;
    refNode->next->prev = node;
    refNode->next = node;
}

//================
//song removal
//================
void songRemv(std::string titleToRemv, SongNode** head) {
    if (*head == nullptr) {
        std::cout << "Nothing to remove" << std::endl;
        return;
    }
    SongNode* current = *head;

    if (current->next == current) {
        if (current->songTitle == titleToRemv) {
            delete current;
            *head = nullptr;
            std::cout << titleToRemv << " was removed" << std::endl;
        } else {
            std::cout << titleToRemv << " was not found in the playlist" << std::endl;
        }
        return;
    }
    do {
        if (current->songTitle == titleToRemv) {
            current->prev->next = current->next;
            current->next->prev = current->prev;

            if (current == *head) {
                *head = current->next;
            }
            delete current;
            std::cout << titleToRemv << " was removed from the playlist" << std::endl;
            return;
        }
        current = current->next;
    } while (current != *head);

    std::cout << titleToRemv << " was not found in the Playlist" << std::endl;
}

//================
//Print all songs in the list
//================
void playAll(SongNode* head) {
    if (head == nullptr) {
        std::cout << "The playlist is empty" << std::endl;
        return;
    }
```

```
    std::cout << "Now Playing Playlist" << std::endl;
    SongNode* current = head;
    int tracknumb = 1;
    do {
        std::cout << tracknumb << " " << current->songTitle << "-" << current->songComposer << std::endl;
        current = current->next;
        tracknumb++;
    } while (current != head);
}

//============================
//song playback control? next, prev
//============================
SongNode* nextSong(SongNode* currentSong) {
    if (currentSong == nullptr) return nullptr;
    return currentSong->next;
}
SongNode* prevSong(SongNode* currentSong) {
    if (currentSong == nullptr) return nullptr;
    return currentSong->prev;
}
void printCurSong(SongNode* currentSong) {
    if (currentSong == nullptr) {
        std::cout << "Nothing to print" << std::endl;
        return;
    }
    std::cout << "Now playing: " << currentSong->songTitle << "-" << currentSong->songComposer << std::endl;
}

//====================
//clearing/cleaning
//====================
void clearPlay(SongNode** head) {
    if (*head == nullptr) return;

    SongNode* tail = (*head)->prev;
    tail->next = nullptr;

    SongNode* current = *head;
    while (current != nullptr) {
        SongNode* temp = current;
        current = current->next;
        delete temp;
    }
    *head = nullptr;
}

#endif //UNTITLED1_PLAYLIST_H
```

Runner Code:

```cpp
#include <iostream>
#include "Playlist.h"
#include <string>


int main() {
    SongNode* playlist = nullptr;   // start with an empty playlist

    // A: build the playlist directly in code
    insSongAtEnd( title: "A Thousand Years", composer: "Cristina Perri", &playlist);
    insSongAtEnd( title: "Heaven Knows", composer: "Orange&Lemons", &playlist);
    insSongAtEnd( title: "Circles", composer: "Post Malone", &playlist);
    insSongAtEnd( title: "Save Your Tears", composer: "The Weeknd", &playlist);

    std::cout << "===== Initial Playlist =====" << std::endl;
    playAll(playlist);

    // B: add a song to the top
    std::cout << "\n===== Adding a song to the TOP =====" << std::endl;
    insSongAtHead( title: "About You", composer: "The 1975", &playlist);
    playAll(playlist);
```

```cpp
    // C: add a song to the middle (after an existing song, found by title)
    std::cout << "\n===== Adding a song to the MIDDLE =====" << std::endl;
    insSongBet( "Heaven Knows", newTitle: "Multo", newComposer: "Cup Of Joe", &playlist); //song in the list -> title-> composer
    playAll(playlist);

    // D: add a song to the end
    std::cout << "\n===== Adding a song to the END =====" << std::endl;
    insSongAtEnd( title: "Lifetime (reimagined)", composer: "Ben&Ben", &playlist);
    playAll(playlist);

    // E: remove a song << user input for a more proper removal
    std::cout << "\n===== Removing a song =====" << std::endl;
    std::string songToRemove;
    std::cout << "Enter the title of the song to remove: ";
    std::getline( [&] std::cin, [&] songToRemove);
    songRemv( songToRemove, &playlist);

    std::cout << "\n===== Playlist after removal =====" << std::endl;
    playAll(playlist);

    // F: step through with next/previous, like a music player's skip buttons
    std::cout << "\n===== Stepping through with next/previous =====" << std::endl;
    SongNode* currentSong = playlist;
    printCurSong(currentSong);
```

```cpp
        std::cout << "Pressing next..." << std::endl;
        currentSong = nextSong(currentSong);
        printCurSong(currentSong);

        std::cout << "Pressing next again..." << std::endl;
        currentSong = nextSong(currentSong);
        printCurSong(currentSong);

        std::cout << "Pressing previous..." << std::endl;
        currentSong = prevSong(currentSong);
        printCurSong(currentSong);

        // G: clear the playlist and free all remaining nodes
        clearPlay(&playlist);
        std::cout << "\n===== Playlist cleared =====" << std::endl;

        return 0;
    }
```

Output:

```
C:\Users\royga\CLionProjects\untitled1\HOA3_SUP_Mendoza.exe
===== Initial Playlist =====
Now Playing Playlist
1 A Thousand Years-Cristina Perri
2 Heaven Knows-Orange&Lemons
3 Circles-Post Malone
4 Save Your Tears-The Weeknd

===== Adding a song to the TOP =====
Now Playing Playlist
1 About You-The 1975
2 A Thousand Years-Cristina Perri
3 Heaven Knows-Orange&Lemons
4 Circles-Post Malone
5 Save Your Tears-The Weeknd

===== Adding a song to the MIDDLE =====
Now Playing Playlist
1 About You-The 1975
2 A Thousand Years-Cristina Perri
3 Heaven Knows-Orange&Lemons
4 Multo-Cup Of Joe
5 Circles-Post Malone
6 Save Your Tears-The Weeknd
```

```
===== Adding a song to the END =====
Now Playing Playlist
1 About You-The 1975
2 A Thousand Years-Cristina Perri
3 Heaven Knows-Orange&Lemons
4 Multo-Cup Of Joe
5 Circles-Post Malone
6 Save Your Tears-The Weeknd
7 Lifetime (reimagined)-Ben&Ben

===== Removing a song =====
Enter the title of the song to remove: Circles
Circles was removed from the playlist
```

```
===== Playlist after removal =====
Now Playing Playlist
1 About You-The 1975
2 A Thousand Years-Cristina Perri
3 Heaven Knows-Orange&Lemons
4 Multo-Cup Of Joe
5 Save Your Tears-The Weeknd
6 Lifetime (reimagined)-Ben&Ben

===== Stepping through with next/previous =====
Now playing: About You-The 1975
Pressing next...
Now playing: A Thousand Years-Cristina Perri
Pressing next again...
Now playing: Heaven Knows-Orange&Lemons
Pressing previous...
Now playing: A Thousand Years-Cristina Perri

===== Playlist cleared =====

Process finished with exit code 0
```

Conclusion:
Provide the following:

- ## Summary of lessons learned:

  > In this lesson, **Linked Lists**, the student was able to understand how to connect things and play and make them run procedurally from node to node. I have learned how to do a **singly linked list**, a type of linked list that only goes to the next and **does not go back**. From my observation, I could use this in the creation of a test or online questions, so that the users would not be able to go back to the past, and do something that they would regret. I also was able to understand the function and the creation of a **doubly linked list**, a type of linked list where the nodes **can go back to the previous node,** not just the next node. This can be used to create like a library or like the activity in the supplementary, a Playlist of songs.

  In essence, this activity could ease and give us an advantage since this is made as our own headers. This lesson, we can apply it to data structures where the data changes very rapidly, since the past sem, we are just doing it basically hard coded, the creation and application of lists and arrays.

- ## Analysis of the procedure:

  > Throughout the procedure, the student first implemented a singly linked list by creating nodes, linking them together, and performing the basic operations of traversal, insertion, and deletion(including the general insertion in the middle of the nodes.). After verifying that each operation produced the expected output, the student modified the implementation into a doubly linked list by adding a **prev** pointer to each node and changing the insertion, traversal, and deletion functions to maintain both **forward and backward** connection.. These modifications demonstrated how a doubly linked list improves navigation by allowing traversal in **both directions** while maintaining the integrity of the list after every iteration and operation. Overall, the procedure provided a practical understanding of dynamic memory allocation, pointer manipulation, and the implementation of linked list operations.

- ## Analysis of the supplementary activity:

  >The supplementary activity helps the student to understand how to creatively create a linked list that would require them to go back(a doubly). The first part, creation and connection of the nodes, was easy enough since this is already done by the professor and would just require some fine tuning or just connecting the variables. The insertions at any part, may it be at the start or the end or the after the sought after song, did provide quite a brief difficulty for me, since it would need me to create a find song function first, so that it would connect the node of the new to the searched song(the in between insertion). Deletion was easy enough but is quite tricky, since this is basically a circle of a list of songs, but quite bearable. But the song Removal form the list is where I found some trouble since, you would have to connect the previous and the next after removing the song, this part is tricky for me, because the first code that I did connected the last song after the removal to the before removed node, like it has been reversed for the 2nd part after the separation.

  > Overall, this supplementary needed much debugging and the knowledge of the concepts and the purpose of each lines to fully understand and not make any errors in the code.

- Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?:

  > In doing the procedure, I do not have any problem since the code was like already given, just some modification, also add the code made by our professor, we are able to do 3.1 to 3.4 very easily

  > As for the supplementary. This took me some time since this requires our own activity and understanding of the given problem. It took me 2 days(Sunday and Monday) to do since I have to refresh my mind often if what I'm doing is proper and correctly follows the need of the activity.

  > And for my feedback on where should I improve…basically, i would need to improve in logic of the creation of the code, I would need to cover all the possible "holes" in the code so that it would run steadily, not justs the de allocation part, but the literal error handling part of the code. As for the coding itself, I do not have a problem since I could easily get some reference from online sources on how it functions or what I need to do. It would still be my own work, since it would require my own interpretation of the code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.